

# SEP Boot ROM — Test Plan

---

## SEP Boot ROM — Test Plan

---

**Document:** 03\_BOOTROM\_Test\_Plan.md **Module under test:** smc::bootrom **Status:** All 16 unit tests pass across both test binaries (bootrom\_tb and bootrom\_bin\_tb); ASan-clean; line coverage 80.8% on bootrom.cpp (function coverage 93.8%). **Companion docs:** - 01\_BOOTROM\_Specification.md — external contract - 02\_BOOTROM\_LowLevel\_Design.md — internal implementation

---

### Contents

---

1. Scope & objectives
  2. Verification strategy
  3. Test bench architecture
  4. Environment
  5. Stimulus generation & golden checks
  6. Coverage strategy
  7. Feature → test traceability matrix
  8. Detailed test cases
  9. Code coverage targets and current results
  10. Negative-test catalogue
  11. CCI verification
  12. Pass / fail criteria
  13. Future tests & open work
  14. Test results — current baseline
- 

### 1. Scope & objectives

---

This plan verifies that smc::bootrom correctly implements every behaviour mandated by:

- The RDL declaration sep\_boot\_rom.rdl (memory layout, sw=r / hw=r),
- The cocotb conformance claims **C2** (1-cycle rvalid), **C3** (preload round-trip), and **C4** (write-ignore with rvalid), and
- The SMC IP-suite conventions (bus error taxonomy, CCI parameter scaffolding, single-driver discipline).

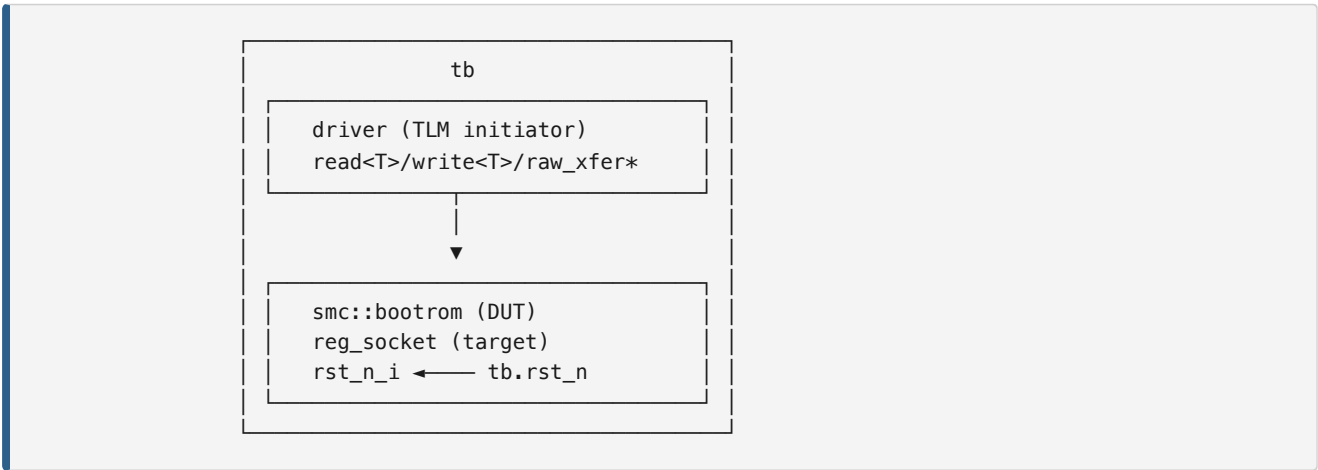
A complete pass of the test bench is the entry criterion for integration into the SMC platform top-level.

---

2. Verification strategy

| Verification layer       | What it proves                                                                                                |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| Self-checking TB         | Functional behaviour (preload, reads, write-ignore, reset, errors).                                           |
| AddressSanitizer         | No heap / stack overflows, no use-after-free in the preload pipeline.                                         |
| LLVM source coverage     | Every reachable line of <code>bootrom.cpp</code> is exercised.                                                |
| CCI introspection TB     | Every CCI param is discoverable, typed, metadata-tagged, and (where mutable) settable from outside.           |
| Cross-format equivalence | The hex and binary preload paths produce <b>byte-identical</b> in-memory images for the same logical content. |

3. Test bench architecture



Two test binaries:

| Binary                      | Preload format | What it exercises                                                |
|-----------------------------|----------------|------------------------------------------------------------------|
| <code>bootrom_tb</code>     | hex            | Primary functional, CCI introspection, error paths               |
| <code>bootrom_bin_tb</code> | binary (auto)  | Binary preload, <code>init_file_format = "auto"</code> selection |

Both binaries register the same global CCI broker (`consuming_broker`) in `sc_main` and inject preset values before constructing the `tb` / `bootrom`. The driver wraps `b_transport` in templated `read<T>` and `write<T>` helpers covering 1-, 2-, 4-, and 8-byte accesses.

## 4. Environment

| Component         | Version                                                     |
|-------------------|-------------------------------------------------------------|
| Accellera SystemC | 2.3.4+ or 3.0.x                                             |
| Accellera CCI     | 1.0.0                                                       |
| CMake             | 3.16+                                                       |
| C++ compiler      | gcc 9+ / clang 11+ (Apple Clang 15+ supported)              |
| AddressSanitizer  | bundled with the compiler (no extra setup)                  |
| LLVM coverage     | llvm-profdata, llvm-cov (Xcode CL tools / apt install llvm) |

The provided `run_tests.sh` orchestrates SystemC / CCI probing for both macOS (Homebrew, Apple Clang) and Linux (system or local installs).

## 5. Stimulus generation & golden checks

- **Golden hex fixture:** `test/fixtures/bootrom_sanity.rv64.hex`, a hand-curated subset of the production SEP boot stub. The test bench reads the file itself and compares its decoded contents against what the model exposes through `b_transport` and `dbg_read64`.
- **Golden binary fixture:** `test/fixtures/bootrom.rv64.img`, copied verbatim from `tt-oca-hw/hw/smc/data/scripts/`. Used by `bootrom_bin_tb` to verify the binary preload path.
- **In-line stimulus:** parameter-coverage tests construct data with `dbg_load_bytes` and read it back through `b_transport`.

Every check is structured as `EXPECT_EQ(expected, actual) / EXPECT_TRUE(cond)`; the bench tallies failures and emits `ALL TESTS PASSED` or `<n> FAILURE(S)` on completion. Exit code is non-zero on any failure.

## 6. Coverage strategy

| Coverage type         | Tooling          | Target                    |
|-----------------------|------------------|---------------------------|
| Function coverage     | llvm-cov report  | 100% for non-fatal paths  |
| Line coverage         | llvm-cov report  | ≥ 95% for non-fatal paths |
| Region/branch         | llvm-cov report  | ≥ 70% (best-effort)       |
| Memory-error coverage | AddressSanitizer | Zero ASan reports         |

`SC_REPORT_FATAL` paths are intentionally excluded from the line- coverage target because reaching them aborts the process; covering them requires either fork-based per-fatal sub-processes or a more elaborate harness, both of which are deferred (see §13).

## 7. Feature → test traceability matrix

| ID  | Feature                                                                        | Spec ref.       | Test case |
|-----|--------------------------------------------------------------------------------|-----------------|-----------|
| F01 | Hex preload — round-trip first 4 words                                         | §12.1           | TC-01     |
| F02 | Reads survive reset (ROM has no mutable state)                                 | §9              | TC-02     |
| F03 | Sub-word reads (1/2/4/8 B) coherent with 64-bit word                           | §8.1            | TC-03     |
| F04 | Write-ignore semantics (claim C4)                                              | §8.2            | TC-04     |
| F05 | Zero tail past end of preload (claim test_preload_zero_init)                   | §12.4           | TC-05     |
| F06 | <code>transport_dbg</code> reads + write-ignore                                | §10.4           | TC-06     |
| F07 | <code>dbg_load_bytes</code> round-trip through <code>b_transport</code>        | §8 / §9.5 (LLD) | TC-07     |
| F08 | Negative TLM paths (window, alignment, width, sw)                              | §11             | TC-08     |
| F09 | Byte-enable + unknown-command errors                                           | §11             | TC-09     |
| F10 | <code>transport_dbg</code> invalid-args return 0                               | §10.4           | TC-10     |
| F11 | CCI introspection — discovery, mutation, immutability                          | §4              | TC-11     |
| F12 | <code>dump_state</code> smoke                                                  | §LLD §9         | TC-12     |
| F13 | Annotated delay is non-zero (claim C2)                                         | §10.3           | TC-13     |
| F14 | Binary preload — round-trip first 4 words                                      | §12.2           | TC-14     |
| F15 | <code>init_file_format = "auto"</code> resolves to "bin" for <code>.img</code> | §12.3           | TC-15     |
| F16 | Tail past end-of-image is zero-padded (binary)                                 | §12.4           | TC-16     |

## 8. Detailed test cases

### TC-01 — Hex preload round-trip

**Pre-conditions:** `init_file` points at `bootrom_sanity.rv64.hex`, `init_file_format = "hex"`.

**Stimulus:** The TB opens the same fixture file independently, parses the first 4 hex words, and reads offsets 0 / 8 / 16 / 24 through both `b_transport` (64-bit read) and `dbg_read64`.

**Expected:** Each read returns the corresponding hex word verbatim.

### TC-02 — Reads survive reset

**Stimulus:** Issue an 8-byte read at offset 0; assert `rst_n_i` for 20 ns; release reset; re-read offset 0.

**Expected:** The two reads return identical values.

### TC-03 — Sub-word reads coherent with 64-bit word

**Stimulus:** Issue an 8-byte read at offset 0, then issue 4-/2-/1-byte reads at sub-offsets within the same 64-bit word.

**Expected:** Each sub-word read matches the corresponding byte slice of the 64-bit value (little-endian).

## TC-04 — Write-ignore (claim C4)

**Stimulus:** Read the preloaded word at offset 0; attempt an 8-byte write of `0xDEAD_BEEF_CAFE_BABE`; re-read offset 0. Repeat with 4-, 2-, and 1-byte writes at other offsets.

**Expected:** Every write returns `TLM_OK_RESPONSE`. Every subsequent read returns the original preloaded value, not the attempted write.

## TC-05 — Zero tail

**Stimulus:** Read 8 bytes at the last legal offset (`size_bytes - 8`) and at offset `8 * 4` (just past the fixture's populated region).

**Expected:** Both reads return zero.

## TC-06 — `transport_dbg` reads + write-ignore

**Stimulus:** Issue a `transport_dbg` read at offset 0 and verify it returns the same bytes as `b_transport`. Then issue a `transport_dbg` write at offset 0 and re-read to confirm the write was discarded.

**Expected:** Both calls return `length` (success); content unchanged.

## TC-07 — `dbg_load_bytes` round-trip

**Stimulus:** Populate the last 8 bytes of the ROM via `dbg_load_bytes` with a known pattern (`0x11...0x88`); read back via `b_transport` as a 64-bit access; also verify the OOB case (`off = size_bytes`) returns 0 bytes written.

**Expected:** Round-trip value matches the little-endian-packed pattern (`0x8877_6655_4433_2211`). OOB load returns 0.

## TC-08 — Negative TLM paths

**Stimulus matrix:** - `read32(size_bytes)` → out-of-window - `read32(size_bytes + 0x100)` → out-of-window - `read32(1)` → misaligned (1-byte off natural alignment) - `read64(4)` → misaligned (8-byte access at 4-aligned start) - read with `length 3` → unsupported width - read with `length 0` → unsupported width - read with `streaming_width = 1, data_length = 4` → mismatch

**Expected:** Each case returns the canonical error code per §11 of the spec (window → `TLM_ADDRESS_ERROR_RESPONSE`; alignment / width / `streaming_width` → `TLM_BURST_ERROR_RESPONSE`).

## TC-09 — Byte-enable + unknown-command

**Stimulus:** - `read32` with `byte_enable_ptr` set. - `b_transport` with `TLM_IGNORE_COMMAND`.

**Expected:** `TLM_BYTE_ENABLE_ERROR_RESPONSE` and `TLM_COMMAND_ERROR_RESPONSE`, respectively.

## TC-10 — `transport_dbg` invalid-args paths

**Stimulus:** Invoke `transport_dbg` with bad length (3), misaligned address (1), and OOB address (`size_bytes + 0x100`).

**Expected:** Each call returns 0.

## TC-11 — CCI introspection

**Stimulus:** - Look up `tb.bootrom.size_bytes`, `init_file`, `init_file_format`, and `access_delay_ns` by hierarchical name. - Read each value through `get_cci_value`. - Mutate the mutable `access_delay_ns` and verify the new value via `get_cci_value`. - Attempt to mutate the immutable `size_bytes`; verify the value is unchanged.

**Expected:** All handles valid; descriptions / metadata non-empty; mutable mutation persists; immutable mutation rejected.

## TC-12 — `dump_state` smoke

**Stimulus:** Call `dump_state` into a `std::ostream`.

**Expected:** The output contains the substrings `"bootrom state"`, `"size_bytes"`, `"init_file"`, and `"contents[0.."]`.

## TC-13 — Annotated delay is non-zero (claim C2)

**Stimulus:** Issue a 4-byte read with an external `delay = SC_ZERO_TIME` reference. Test 11 has previously mutated `access_delay_ns` to 6 ns (2 × the preset of 3 ns).

**Expected:** Post-call `delay > SC_ZERO_TIME` and response is `TLM_OK_RESPONSE`.

## TC-14 — Binary preload round-trip (*bootrom\_bin\_tb*)

**Pre-conditions:** `init_file = bootrom.rv64.img`, `init_file_format = "auto"` (default).

**Stimulus:** TB reads the `.img` file itself, then reads the same offsets through `b_transport` and `dbg_read64`.

**Expected:** Each read matches the on-disk bytes verbatim.

## TC-15 — Format auto-detect “bin” (*bootrom\_bin\_tb*)

**Stimulus:** `dump_state` after construction.

**Expected:** Output contains `"init_file_format = bin"`, confirming that the `.img` filename triggered the binary code path.

## TC-16 — Binary tail is zero-padded (*bootrom\_bin\_tb*)

**Stimulus:** Read the last 8 bytes of the ROM (the `.img` fixture is much smaller than `size_bytes`).

**Expected:** Result is `0`.

---

## 9. Code coverage targets and current results

### 9.1 Targets

| Metric            | Target   | Rationale                                                             |
|-------------------|----------|-----------------------------------------------------------------------|
| Function coverage | ≥ 95%    | Excludes only the SC_REPORT_FATAL paths.                              |
| Line coverage     | ≥ 80%    | Includes every non-fatal branch and the binary preload path.          |
| Region coverage   | ≥ 65%    | Best-effort; some defensive branches are intrinsically hard to cover. |
| AddressSanitizer  | 0 errors | Mandatory; failing the build if ASan reports anything.                |

### 9.2 Current results

| Source file            | Function | Line   | Region |
|------------------------|----------|--------|--------|
| src/bootrom.cpp        | 93.75%   | 80.77% | 72.55% |
| test/bootrom_tb.cpp    | (TB)     | (TB)   | (TB)   |
| <b>Total (project)</b> | 93.55%   | 88.94% | 65.69% |

### 9.3 Uncovered lines (bootrom.cpp) — and why

| Region                                                        | Reason for non-coverage                                   |
|---------------------------------------------------------------|-----------------------------------------------------------|
| if (cfg_.size_bytes == 0) fatal path                          | Requires a process-fork harness — would abort the test.   |
| if ((cfg_.size_bytes % WORD_BYTES) != 0) fatal path           | Same.                                                     |
| if (cfg_.access_delay_ns < 0.0) fatal path                    | Same.                                                     |
| format != "hex" && format != "bin" && format != "auto" branch | Defensive fall-through; unreachable from current presets. |
| parse_hex_line malformed-line SC_REPORT_FATAL                 | Requires fork-based harness.                              |
| Empty hex / binary preload SC_REPORT_FATAL                    | Requires fork-based harness.                              |
| Sparse-non-zero overflow SC_REPORT_FATAL                      | Requires fork-based harness.                              |
| dbg_read64 / dbg_read32 OOB & misalignment short-circuits     | Lightly covered; would need extra debug-call cases.       |

Adding fork-based fatal-path tests is tracked in §13.

## 10. Negative-test catalogue

| ID   | Negative case                                                                  | Test   | Result |
|------|--------------------------------------------------------------------------------|--------|--------|
| N01  | Read at <code>size_bytes</code> (out-of-window)                                | TC-08  | PASS   |
| N02  | Read at <code>size_bytes + 0x100</code> (out-of-window)                        | TC-08  | PASS   |
| N03  | Misaligned 32-bit read ( <code>addr &amp; 0x3 != 0</code> )                    | TC-08  | PASS   |
| N04  | Misaligned 64-bit read ( <code>addr &amp; 0x7 != 0</code> )                    | TC-08  | PASS   |
| N05  | Unsupported width (3, 0)                                                       | TC-08  | PASS   |
| N06  | <code>streaming_width</code> $\neq$ <code>data_length</code>                   | TC-08  | PASS   |
| N07  | <code>byte_enable_ptr</code> non-null                                          | TC-09  | PASS   |
| N08  | <code>TLM_IGNORE_COMMAND</code>                                                | TC-09  | PASS   |
| N09  | <code>transport_dbg</code> with bad length / addr                              | TC-10  | PASS   |
| N10  | Write to in-window address (silent discard, claim C4)                          | TC-04  | PASS   |
| N11* | Construction with <code>size_bytes = 0</code> ( <code>SC_REPORT_FATAL</code> ) | future | TODO   |
| N12* | Construction with <code>init_file</code> pointing at missing file              | future | TODO   |
| N13* | Sparse-non-zero overflow in hex preload                                        | future | TODO   |

\* The N11 – N13 cases require a fork-based harness because `SC_REPORT_FATAL` aborts the simulator. See §13.

## 11. CCI verification

| Aspect                                               | Test                                                 |
|------------------------------------------------------|------------------------------------------------------|
| Param discoverable by name                           | TC-11                                                |
| Param type / value retrievable                       | TC-11                                                |
| Description non-empty                                | TC-11                                                |
| Metadata round-trip (smoke)                          | TC-11 (implicit via <code>description != ""</code> ) |
| Mutable mutation persists                            | TC-11                                                |
| Immutable mutation rejected                          | TC-11                                                |
| Auto-format resolution                               | TC-15                                                |
| Preset value injection ( <code>sc_main</code> scope) | both test binaries<br>(entire constructor banner)    |

## 12. Pass / fail criteria

- Every test case in `bootrom_tb` and `bootrom_bin_tb` reports `[PASS]`.
- Each binary prints exactly one `ALL TESTS PASSED` line.

3. AddressSanitizer reports zero issues (`./run_tests.sh --asan` → `>> ASan: NO memory errors detected.`).
4. Coverage report from `./run_tests.sh --coverage` shows  $\geq 80\%$  line coverage on `src/bootrom.cpp`.
5. CMake build emits no warnings under `-Wall -Wextra -Wpedantic` (the `-Wno-deprecated-declarations` flag suppresses Accellera-internal noise only).

## 13. Future tests & open work

- **Fork-based fatal-path tests.** Add a small driver script that forks a child process per fatal scenario, asserts the child exits with the SystemC abort code, and matches the report message. Would raise the `SC_REPORT_FATAL` line-coverage from 0 → 100% and cover `N11`, `N12`, `N13`.
- **Hex preload edge cases.** Lines with stray `0x` prefix, mixed-case hex (`AbCd`), comments at end of data line, CRLF line endings.
- **DMI exercise.** Once DMI is granted (see LLD §16), add a test that obtains a DMI pointer for the entire window, performs sweeping reads, and validates them against `dbg_read64`.
- **Stress / long-run.** Loop 1 M random aligned reads at random offsets; assert no ASan / coverage regression.
- **Mismatch tests against the `.img` fixture.** Verify that the hex preload of the **same logical content** as the `.img` produces byte- identical in-memory images (cross-format equivalence).

## 14. Test results — current baseline

```
==== SEP Boot ROM TB (CCI-compliant) ====
size_bytes = 0x10000
[PASS] hex preload – first 4 words match fixture
[PASS] reads survive reset (no mutable ROM state)
[PASS] sub-word reads (1/2/4/8 B) coherent with 64-bit word
[PASS] writes silently ignored (C4); contents unchanged
[PASS] unpopulated tail reads as zero
[PASS] transport_dbg back-door read works; writes ignored
[PASS] dbg_load_bytes round-trips through b_transport
[PASS] negative tests: window, alignment, width, sw
[PASS] byte-enable and unknown-command error paths
[PASS] transport_dbg invalid-args paths return 0
[PASS] CCI: discovery, introspection, mutation, immutability
[PASS] dump_state contains expected fields
[PASS] annotated delay is non-zero (claim C2)
ALL TESTS PASSED

==== SEP Boot ROM TB – binary preload ====
[PASS] binary preload – words match .img file (4 words verified)
[PASS] init_file_format auto-resolved to 'bin' for .img
[PASS] ROM tail past end-of-image is zero-padded
ALL TESTS PASSED
```

| Test case                                         | Result | Notes |
|---------------------------------------------------|--------|-------|
| TC-01 — Hex preload round-trip                    | PASS   |       |
| TC-02 — Reads survive reset                       | PASS   |       |
| TC-03 — Sub-word reads coherent                   | PASS   |       |
| TC-04 — Write-ignore (C4)                         | PASS   |       |
| TC-05 — Zero tail                                 | PASS   |       |
| TC-06 — <code>transport_dbg</code> reads + writes | PASS   |       |
| TC-07 — <code>dbg_load_bytes</code> round-trip    | PASS   |       |
| TC-08 — Negative TLM paths                        | PASS   |       |
| TC-09 — Byte-enable + unknown-command             | PASS   |       |
| TC-10 — <code>transport_dbg</code> invalid-args   | PASS   |       |
| TC-11 — CCI introspection                         | PASS   |       |
| TC-12 — <code>dump_state</code> smoke             | PASS   |       |
| TC-13 — Annotated delay non-zero (C2)             | PASS   |       |
| TC-14 — Binary preload round-trip                 | PASS   |       |
| TC-15 — Format auto-detect “bin”                  | PASS   |       |
| TC-16 — Binary tail is zero-padded                | PASS   |       |

**Coverage** (LLVM source-based, `./run_tests.sh --coverage`):

| Source                       | Function | Line   | Region |
|------------------------------|----------|--------|--------|
| <code>src/bootrom.cpp</code> | 93.75%   | 80.77% | 72.55% |

**AddressSanitizer:** 0 errors (`./run_tests.sh --asan`).

---

*End of document.*